

Weird (and maybe useful) pwn techniques

@Chino

25/07/2023

ret2csu

ret2csu

- How to use `__libc_csu_init` gadgets
- (if compiled with) `libc < 2.34`
- Binary leak or no PIE
- Function pointers (GOT?)

ret2csu

401220:	4c 89 f2	caller: mov	rdx, r14
401223:	4c 89 ee	mov	rsi, r13
401226:	44 89 e7	mov	edi, r12d
401229:	41 ff 14 df	call	QWORD PTR [r15+rbx*8]
40122d:	48 83 c3 01	add	rbx, 0x1
401231:	48 39 dd	cmp	rbp, rbx
401234:	75 ea	jne	401220 <__libc_csu_init+0x40>
401236:	48 83 c4 08	add	rsp, 0x8
40123a:	5b	popper: pop	rbx
40123b:	5d	pop	rbp
40123c:	41 5c	pop	r12
40123e:	41 5d	pop	r13
401240:	41 5e	pop	r14
401242:	41 5f	pop	r15
401244:	c3	ret	

ret2dlresolve

ret2dlresolve

- Make the binary to resolve an arbitrary function and call it
- Binary leak or no PIE
- Call only functions under libc symbol
- Need to control argument's registers
- Need a read at specific address

SROP

SigReturn Oriented Programming

- Syscall to return from a signal interrupt
- Pops the “saved” state (all the registers and something more) from the stack
- We need to set every register (also rip and rsp), hence we need leak or no PIE
- We need a syscall (and rax control)
- We need a big overflow (0x100 bytes)

BROP

BROP

- Blind ROP
- You don't have the binary
- Compile a small binary to have some references
- Average BROP challenge:

```
void fun() {  
    char buf[0x10];  
    gets(buf);  
}  
int main() {  
    puts("This is a stupid challenge");  
    fun();  
    puts("Bye");  
}
```

BROP

1. Brute force BOF offset

- Send “A” * *i* until something different happens (segfault message, no print after read, connection instantly closed, ...)
- Be careful about rbp, the overflow offset might be 8 / 4 bytes after what you found

BROP

2. Find arch and PIE / no PIE

- First brute force plt (64 bit: *0x4010xx*, 32 bit: *0x80490xx*, PIE: only last byte)
- If pie and last byte doesn't work, you can bruteforce two bytes, be careful that the 4th nibble is affected by *aslr*
- If PIE you need a way to leak / chain stuffs
- You might feel super lucky and hope to be on the return address of main and bruteforce a one gadget

BROP

3. Find a print

- Hope that registers are good and brute force PLT, then, if it doesn't work, the whole binary
- Find a print in the binary and try the next step with addresses around that

0x1000 print

0x1001 print

0x1002 print

0x1003 no print

0x1004 no print

0x1005 no print

BROP

4. Find a gadget

- Gadget + something printable + print
- (guessed gadget) * 2 + print
- You can add multiple time the gadget in the hope to find gadgets like pop rdi; pop rsi; ret;
- If you feel wild enough, you can try to guess the csu gadgets addresses

BROP

5. Dump the binary

- Now You should have an arbitrary read primitive
- Dump the whole binary and pwn it

ret2vDSO

ret2vDSO

- Kernel dependent area
- Brute-forceable before Linux 3.18.2 (11 bits)
- Few gadgets and syscall